

Model-based Reinforcement Learning via Predictive Sampling

Thiago da Cunha Vasco

April, 2024

Abstract

This report details my course project for CPSC 533V, for which I experimented with a rudimentary form of model-based reinforcement learning. The basic idea for the algorithm is to train a neural network to learn the dynamics of an MDP, and then via sampling choose an optimal action sequence which (approximately) maximizes the future return over a finite time horizon. As is typical of MPC, a brand-new action sequence is computed at each time step, but only the first action in the entire sequence is acted upon. The GitHub repository for this project is available [here](#).

1 Introduction

Model-based reinforcement learning (MBRL) is a subset of reinforcement learning where instead of directly learning a policy or action-value function, the agent learns a model of its environment and uses it to compute an optimal action at each timestep. The main goal of my project is to successfully control a 3D robot in MuJoCo using MBRL, where the dynamics is modeled using a neural network and the agent uses MPC as its policy. This project report contains five sections; the second section outlines the foundations of the MBRL algorithm I implemented, the third section delves into the implementation issues I ran into and their fixes, the fourth section shows empirical results, and the last section will summarize some of my takeaways from working on this project.

2 Methodology

2.1 Reinforcement Learning

For this project, I apply MBRL to find a satisfactory solution to the MuJoCo Ant-v4 task. Generally, reinforcement learning tasks can be formulated as a Markov Decision Process $(S, A, \mathbb{P}, R)$ where the S is the set of all states, A is the set of all actions, $\mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a)$ is the transition dynamics, and $R(s_t, a_t)$ is the reward function. For this project, we can assume that the dynamics are deterministic rather than stochastic, and is represented by some unknown function $f : S \times A \rightarrow S$. The goal of reinforcement learning in general is to find a policy π which maximizes

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \quad (1)$$

such that $a_t \sim \pi(s_t)$, $s_0 \sim p(s_0)$, and $\gamma \in [0, 1]$. Since we're assuming the dynamics is fully deterministic, we can do away with the expectation and restate the objective as

$$J(\pi; s_0) = \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \quad (2)$$

where $s_{t+1} = f(s_t, a_t)$ and $a_t = \pi(s_t)$ at any time t . Here we take the initial state s_0 as an input.

2.2 Model-Based Reinforcement Learning

2.2.1 Learning the Dynamics Model

Instead of directly estimating a policy or value function, MBRL attempts to learn the dynamics model f . For this project, I trained a neural network with two hidden layers (each containing 512 nodes) and ReLU activations to learn a model of the dynamics $f_\theta(s_t, a_t) = s_{t+1} - s_t$. The reason why f_θ is set-up to predict the increment in states $s_{t+1} - s_t$ rather than just s_{t+1} is because when Δt is small, subsequent states are highly similar which makes a_t have seemingly little effect on the dynamics. This makes the dynamics model difficult to learn. This issue is circumvented by instead predicting increments [8].

2.2.2 Replay Buffer

During training, rollouts $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$ are generated by applying the MPC policy to the environment. These trajectories are then chopped up into tuples of the form (s_t, a_t, s_{t+1}) and stored into a replay buffer D . After each episode, we learn the dynamics model by minimizing the objective

$$\frac{1}{|D|} \sum_{(s_t, a_t, s_{t+1}) \in D} \frac{1}{2} \|(s_{t+1} - s_t) - f_\theta(s_t, a_t)\|_2^2 \quad (3)$$

using Adam. Note that before tuples are stored in the replay buffer, states and actions are normalized to have mean zero and a standard deviation of 1. A small amount of Gaussian noise is also applied to the collected data in order to improve the robustness of the learned dynamics model. One important detail is that the replay buffer D is in truth a union of two other replay buffers D_{rand} and D_{train} . Because our MBRL algorithm learns solely by training f_θ on (s_t, a_t, s_{t+1}) tuples, training can be done using off-policy data. Thus at the beginning of training, we can kick-start the learning process by training f_θ on action sequences that are completely randomly generated for N_{init} episodes. Rollout data generated by these random action sequences are stored in D_{rand} . Once we've completed the first N_{init} episodes, we then use the sampling-based MPC as our policy and the new tuples are stored in D_{train} for the rest of training. The dynamics f_θ is then trained on batches where $(100 \times \alpha_{\text{train}})\%$ of the data comes from D_{train} and the rest comes from D_{rand} . We denote the total number of episodes in training as N_{train} .

2.2.3 The policy

The agent will find an optimal action by using an MPC at each time step. Let H be the prediction horizon of the MPC controller. The MPC works by finding an optimal sequence of actions $a_t, \dots, a_{t+H-1}$ at each time step t which maximizes $\sum_{k=0}^{H-1} \gamma^k R(s_{t+k}, a_{t+k}) + \phi(s_{t+H})$ where $s_{t'+1} = s_{t'} + f_\theta(s_{t'}, a_{t'})$ and $\phi(\cdot)$ denotes some terminal reward function. After finding the optimal sequence, only the first action a_t is actually applied to the system. We repeat this for each new time step. In order to solve this optimization problem, I made use of predictive sampling. In predictive sampling, you randomly generate K candidate action sequences by adding zero-mean Gaussian noise to the entirety of the optimal sequence computed in the previous time step (the optimal action sequence of subsequent time steps tend to be similar). Then for each of those K sequences, compute each of their respective returns, and take the first action belonging to the sequence with the highest assigned return [7].

Algorithm 1: Predictive Sampling MPC

Data: `state_0` (the current state), `past_seq` (previous optimal action sequence),
`n_action` (size of action vector), `R` (reward function), `fθ` (dynamics function)

```
1 rets  $\leftarrow$  zeros(K);
2 action_seq  $\leftarrow$  zeros(H * n_action, K);
3 for i = 0, ..., K - 1 (in parallel) do
4   action_seq[:, i]  $\leftarrow$  past_seq +  $\mathcal{N}(0, \sigma^2 * I)$ ;
5   state  $\leftarrow$  state_0;
6   for t = 0, ..., H - 1 do
7     action  $\leftarrow$  action_seq[i, t * n_action : (t + 1) * n_action];
8     rets[i]  $\leftarrow$  rets[i] +  $\gamma^t * R(\text{state}, \text{action})$ ;
9     if episode terminates at state-action pair (state, action) then
10      | break;
11    end
12    state  $\leftarrow$  fθ(state, action) + state;
13  end
14  rets[i]  $\leftarrow$   $\phi(\text{state})$ ;
15 end
16 opt_idx  $\leftarrow$  argmax(rets);
17 return action_seq[:, opt_idx];
```

3 Implementation

3.1 Predictive Sampling vs Gradient-Based Solvers

I originally had planned to use a gradient-based solver so that exact solutions for the MPC can be found, and the Python library package `do-mpc` was suggested to me by colleagues as a possible way of achieving this [2]. The complication of using a gradient-based solver in this context comes from the fact that our dynamics function is being modeled by a neural network with an activation function that is non-differentiable at zero. This issue of non-differentiability within the network may have been the reason why the `do-mpc` solver failed to converge under ReLU activations. This problem was solved by switching out the ReLU activations for tanh activations, but then another problem emerged: the runtime of the solver would take substantially longer the larger I made the network. Since I wanted to be able to run MuJoCo simulations in real-time, and since I wanted a network that was sufficiently expressive, I opted for the predictive sampling method instead.

3.2 Optimizing Code for Predictive Sampling

Although conceptually easy, predictive sampling can be significantly more expensive than gradient-based methods due to the fact that at each time step, returns for each K generated trajectories will have to be computed. As H is our prediction horizon, this entails having to invoke a forward pass on our network $K \cdot H$ times at each time step. For the Ant-v4 task, I set $K = 1024$ and $H = 15$, which implies $K \cdot H = 15360$ forward passes at each time step! This resulted in me having to spend four to five days optimizing my code in order to get MuJoCo environments running anywhere near real time.

3.2.1 Multiprocessing

The largest performance boost came from parallelizing my software into multiple processes. In order to implement multiprocessing, I used Ray which is an open source compute platform used for speeding up

Algorithm 2: MBRL Training

Data: N_{rand} , N_{train} , ep_len (max length of episode), max_iter (max number of episodes)

```
1 Construct empty lists  $D_{\text{rand}}$  and  $D_{\text{train}}$ ;
2 Randomly initialize  $f_{\theta}$  network;
3 Generate dataset  $D_{\text{rand}}$  by randomly sampling actions for  $N_{\text{rand}}$  episodes;
4 for  $\text{ep} = 0, \dots, \text{max\_iter} - 1$  do
5   Generate agent's initial state  $s_0$ ;
6   state  $\leftarrow s_0$ ;
7   for  $t = 0, \dots, \text{ep\_len} - 1$  do
8     Let  $\text{opt\_actions}$  be the optimal action sequence returned by Algorithm 1;
9     action  $\leftarrow \text{opt\_actions}[0 : n_{\text{action}}]$ ;
10    Step the environment to get next_state;
11    Append (state, action, next_state) to  $D_{\text{train}}$ ;
12    state  $\leftarrow \text{next\_state}$ ;
13  end
14  Train  $f_{\theta}$  on objective 3 using  $D_{\text{rand}}$  and  $D_{\text{train}}$ ;
15 end
16 return  $f_{\theta}$ ;
```

workloads [3]. The Python Ray library provides a clean and convenient way to asynchronously complete multiple tasks on separate workers.

One of the challenges of multiprocessing is that the time it takes to onboard a new task onto a worker can seriously hamper performance gains; this problem is especially acute when each task is somewhat short (as is computing the return of a single trajectory). Let T be the total number of tasks, let c_{task} be the amount of time it takes to complete a given task (we're assuming that every task has an identical runtime), and let c_{on} be the amount of time it takes to onboard a new task onto a worker. Note that in our case, $T = K$ and c_{task} is equal to the amount of time needed to compute a single forward pass. Suppose N_{cores} is the number of cores our machine has. Without multiprocessing our algorithm will take $c_{\text{task}} \cdot T$ time, and with multiprocessing it will take $\frac{c_{\text{task}} \cdot T}{N_{\text{cores}}} + c_{\text{on}} \cdot T$ time, and so our speed up is

$$\frac{c_{\text{task}} \cdot T}{\frac{c_{\text{task}} \cdot T}{N_{\text{cores}}} + c_{\text{on}} \cdot T} = \frac{1}{\frac{1}{N_{\text{cores}}} + \frac{c_{\text{on}}}{c_{\text{task}}}}. \quad (4)$$

Since the amount of cores we have on our machine is typically fixed, the only way we can improve (1) is by making $c_{\text{on}} \ll c_{\text{task}}$. The Ray library allows for multiprocessing in a way that significantly reduces the amount of time it takes to pass data to workers, which results in a significantly smaller c_{on} . Furthermore, multiprocessing can be implemented in a way such that tasks are “batched” together; i.e., one worker process is given all the data necessary to work on several tasks back-to-back without having to reboot itself for each new task. In my specific implementation, I batched together $\frac{T}{N_{\text{cores}}} = \frac{1024}{16} = 64$ tasks per worker. This results in a runtime of $\frac{c_{\text{task}} \cdot T}{N_{\text{cores}}} + c_{\text{on}} \cdot N_{\text{cores}}$ and consequently gives us a speedup of

$$\frac{c_{\text{task}} \cdot T}{\frac{c_{\text{task}} \cdot T}{N_{\text{cores}}} + c_{\text{on}} \cdot N_{\text{cores}}} = \frac{1}{\frac{1}{N_{\text{cores}}} + \frac{c_{\text{on}}}{c_{\text{task}}} \cdot \frac{N_{\text{cores}}}{T}}. \quad (5)$$

Since $T \gg N_{\text{cores}}$, this greatly improves the runtime of our algorithm.

3.2.2 Efficient Matrix Multiplication

Although parallelization significantly improved things, I still managed to find easy ways of speeding up my predictive sampling algorithm. Recall that at each time step, our machine needs to compute around 15360 forward passes. This suggests that it would be worthwhile investigating how we can speed up all of those matrix multiplications (and hence reduce c_{task}). According to a comment made by a PyTorch developer, forward passes on small networks can incur a small amount of overhead which can actually make it more efficient to compute these forward passes outside of the PyTorch framework [1]. I therefore converted the tensor arrays of my network into NumPy arrays, and computed the matrix multiplication directly using BLAS functions from `scipy.blas`. This led to modest improvements in runtime.

4 Experiments

4.1 The Environment

I experimented with the MBRL algorithm on the MuJoCo Ant-v4 task, and used my own custom reward function

$$R(s, a) = s^{XVEL} + 0.5 - \frac{1}{16} \|a\|_2^2 \quad (6)$$

which rewards the agent for moving the ant in the direction of the positive x-axis as quickly as possible, while also punishing the agent for using control inputs which are too extreme.

4.2 Results

Here I present a couple of empirical results obtained from the MBRL algorithm on the Ant-v4 task. Figure 1 illustrates returns achieved by MBRL across training episodes. Note that this data was collected across five separate training runs, with $K = 1024$, $H = 15$, $\gamma = 0.99$, and episode lengths of 200 time steps. Each run consisted of 275 episodes, which amounts to about 45 simulated minutes. While the first 25 episodes of each run solely used random actions, the following 250 episodes used an epsilon-greedy strategy where at each time step, there is a 95% chance of MBRL outputting an MPC action, and a 5% chance of outputting a random action. This is done to encourage exploration. The algorithm appears to steadily improve throughout training, although some degradation in performance appears to occur near the end of it. Note the sudden dip in minimum return around episode 200: at that particular time step one of the evaluation rollouts was terminated early (hence the low return). I found that low returns later in training usually occurs when the ant jumps too high and causes the episode to terminate early.

Another experiment that I ran was changing MPC hyperparameters to see how that would affect performance. On a trained dynamics model I ran 10 experiments for each unique setting of (H, K) , and results are shown in Figure(s) 2(a) and 2(b). One would expect that the performance of the MBRL algorithm improves as K increases, and Figure 2(b) seems to support this intuition up until $K = 2048$ which yields the worst performance (this is probably sampling error since theoretically more trajectories should bring us closer to optimal behavior). Meanwhile, Figure 2(a) proves MPC to be a poor policy when H is too small, while also showing that making H too large leads to diminishing returns.

5 Conclusion

The first takeaway from this project is that MBRL makes a tradeoff between efficiency in training and efficiency in inference. MBRL in general is known for having better sample efficiency than model-free methods, and my experience confirms this to be the case. In just 50 minutes worth of training data, the simple algorithm I described in section 2 is capable of generating a not-so-terrible solution to the Ant-v4 task.

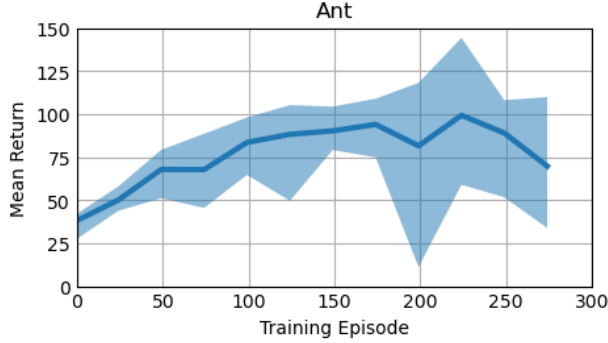


Figure 1: Returns achieved throughout training. The dark-blue line represents the return averaged across all runs, whereas the upper- and lower-bounds of shaded light-blue area represents the minimum and maximum return achieved across all runs at a given time step.

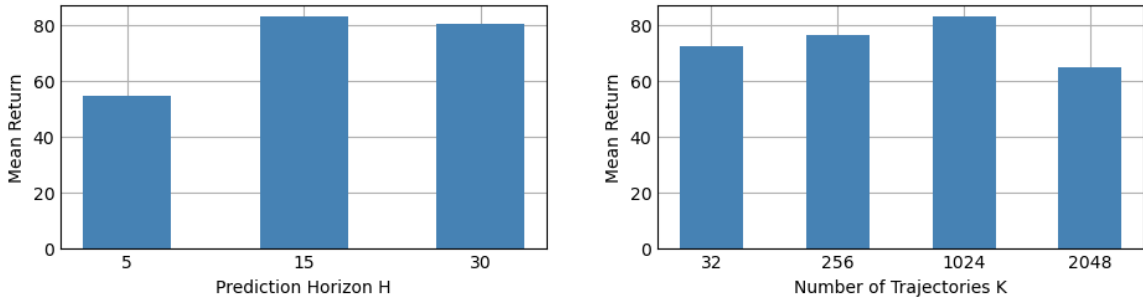


Figure 2: (a) Other parameters are $K = 1024, \gamma = 0.99$. The performance of the MBRL algorithm degrades when H is set too low. (b) Other parameters are $H = 15, \gamma = 0.99$. The performance of the MBRL algorithm steadily improves as K increases (with the exception of $K = 2048$).

One thing to note however, is that although model-based methods are more sample efficient during training, they are usually more computationally expensive during inference than their model-free counterparts. This is especially true if you're using a predictive sampling scheme like the one I implemented. As for discussing the fluidity of the motion generated by the MBRL approach, model-based methods tend to underperform at generating natural motions relative to their model-free counterparts. In tandem with [6], one can think of how biological systems move to see why this might be the case. It turns out that animal locomotion is mostly habitual and subconscious, and not methodically planned. To give an analogy, if I were to stand up from my chair and walk to my fridge, the intricate ways in which I would move my muscles in order to complete these tasks is deeply ingrained within me and automatic, and not the product of conscious planning. This is due to the fact that calculating an entire optimal action sequence at each timestep is extremely expensive for biological systems, given the complexity of the dynamics of our bodies and the environment around us. Thus, attempting to generate natural movement through planning on an imperfect dynamics model is not only expensive, but also produces sporadic and unnatural movement.

References

- [1] Why is my numpy implementation 8x faster than pytorch forward pass? <https://discuss.pytorch.org/t/why-is-my-numpy-implementation-8x-faster-than-pytorch-forward-pass/45459>, 2019.
- [2] <https://www.do-mpc.com/en/latest/>, 2024.
- [3] <https://www.ray.io/>, 2024.
- [4] <https://github.com/Thiagodcv/predictive-sampling-mpc>, 2024.
- [5] Timothée Anne, Jack Wilkinson, and Zhibin Li. Meta-learning for fast adaptive locomotion with uncertainties in environments and robot dynamics. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4568–4575. IEEE, 2021.
- [6] Nicole Drummond and Yael Niv. Model-based decision making and model-free learning. *Current Biology*, 30(15):R860–R865, 2020.
- [7] Taylor Howell, Nimrod Gileadi, Saran Tunyasuvunakool, Kevin Zakka, Tom Erez, and Yuval Tassa. Predictive sampling: Real-time behaviour synthesis with mujoco. *arXiv preprint arXiv:2212.00541*, 2022.
- [8] Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, and Sergey Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. *CoRR*, abs/1708.02596, 2017.